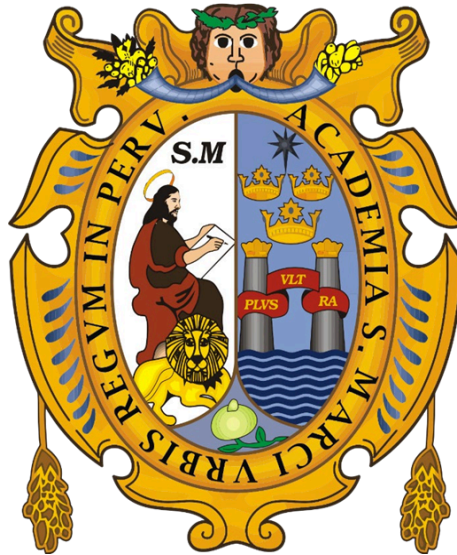


**UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS**  
**FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA**  
**E.P DE INGENIERÍA DE SOFTWARE**



***Gestión de la configuración de software***

Plan de gestión de la configuración de software

**DOCENTE:**

Dra. Marlene Reyes

**ALUMNOS:**

Bohorquez Huaranga Kevin Rhamses

León Chacón Gerardo Raúl

Lopez Leonardo

Rojas Rojas Max

Sanchez Gotea Edu Joseph

Sevan Reyes David

Taco Zavala Miguel Angel

Vera Rodriguez Cristian Leonardo

# **Plan de gestión de la configuración de software**

**Patitas Sanas**

**Versión 1.0.0**

## **Historia de revisiones**

| Fecha      | Versión | Descripción                    | Autor       |
|------------|---------|--------------------------------|-------------|
| 07/05/2026 | v1.0.0  | Creación inicial del documento | Edu Sanchez |
|            |         |                                |             |
|            |         |                                |             |

## Índice

|                                                   |    |
|---------------------------------------------------|----|
| 1. Introducción y Alcance                         | 3  |
| 2. Organización y Responsabilidades               | 4  |
| 2.1. Estructura organizacional                    | 4  |
| 2.2. Roles del equipo                             | 4  |
| 3. Actividades de Gestión de Configuración        | 6  |
| 3.1. Identificación de la Configuración           | 6  |
| 3.1.1. Catálogo de ECS del Proyecto Patitas Sanas | 7  |
| 3.1.2. Nomenclatura de Versiones                  | 8  |
| 3.1.3 Estructura del Repositorio                  | 8  |
| 4. Cronograma y Líneas Base (Baselines)           | 11 |
| 5. Herramientas y Entorno                         | 13 |
| 5.8. Comunicación y Reportes de Estado            | 15 |
| Referencias                                       | 16 |

## 1. Introducción y Alcance

### 1.1. Introducción

El presente documento define el Plan de Gestión de la Configuración de Software (PGCS) para el proyecto Patitas Sanas, un sistema de gestión clínica veterinaria.

En un entorno de desarrollo colaborativo es fundamental establecer mecanismos que garanticen la integridad, trazabilidad y control de cambios en los productos de software generado teniendo eso en cuenta este plan detalla las normativas, herramientas y procesos que permitirán que el código fuente, la documentación y los entornos de ejecución evolucionen de forma organizada.

### 1.2. Propósito

El propósito de este plan es proporcionar una base sólida para el Control de los Elementos de Configuración de software (ECS). Se busca minimizar los riesgos derivados de cambios no autorizados, asegurar que el equipo trabaje sobre versiones validadas y facilitar la auditoria del ciclo de vida del proyecto.

### 1.3. Alcance

#### 1.3.1. Desarrollo de Software (Código Fuente)

- **Backend:** Implementación de la lógica de negocio utilizando el framework Fast API en Python. Incluye módulos para el registro de mascotas, historiales clínicos, gestión de veterinarios, especialidades, turnos de trabajo, citas y un sistema de autenticación por roles.
- **Frontend:** Implementación de la lógica de negocio utilizando el framework React + Vite. Incluye módulos para el registro de mascotas, historiales clínicos, gestión de veterinarios, especialidades, turnos de trabajo, citas y un sistema de autenticación por roles.

#### 1.3.2. Gestión de Datos

Diseño y mantenimiento de una base de datos relacional en MySQL. Esto incluye los scripts de creación de esquemas (DDL), la carga de datos semilla (DML) y la gestión de migraciones para asegurar la evolución estructural del sistema.

#### 1.3.3. Infraestructura y Despliegue

Despliegue del servidor y de la base de datos en Railway. Despliegue del frontend en Vercel

#### 1.3.4. Documentación

Elaboración del Documento de Requerimientos del Software (SRS), el Plan de Gestión de Configuración y manuales detallados de usuario y despliegue además de un seguimiento del proyecto mediante la app Jira.

#### 1.4. Glosario

- **ECS (Elemento de Configuración de Software):** Cualquier pieza de software, documentación o datos que deba ser controlada.
- **CCB (Configuration Control Board):** Comité de Control de Cambios encargado de evaluar y aprobar modificaciones.
- **SemVer:** Estándar de Versionado Semántico utilizado para la nomenclatura de versiones.
- **Baseline (Línea Base):** Especificación o producto que ha sido revisado formalmente y sobre el cual se basará el desarrollo posterior.

## 2. Organización y Responsabilidades

### 2.1. Estructura organizacional

El proyecto estará conformado por estudiantes de la carrera de ingeniería de software de séptimo ciclo de la Facultad de Ingeniería de Sistemas e Informática (FISI) de la Universidad Nacional Mayor de San Marcos (UNMSM) que se encargaran de las diferentes etapas del desarrollo de software, incluyendo planificación, análisis, diseño, implementación, pruebas y mantenimiento.

La estructura organizacional del proyecto se presenta de la siguiente manera

- Cliente / Usuario Final
- Jefe de Proyecto
- Analista de Sistemas
- Equipo de Desarrollo
- Equipo de Pruebas y Calidad
- Soporte Técnico

El flujo de comunicación y supervisión será jerárquico, donde el Jefe de Proyecto coordinará las actividades generales y supervisará el cumplimiento de los objetivos establecidos.

## 2.2. Roles del equipo

| ROL                                                    |                                   | FUNCION                                                                                                                                                                                                                                                                                                 | RESPONSABILIDAD                                                                                                                                                                                                                                           |
|--------------------------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jefe de Proyecto - Vera Rodriguez                      |                                   | <ul style="list-style-type: none"> <li>- Dirigir y supervisar el desarrollo general del proyecto.</li> <li>- Coordinar las actividades del equipo de trabajo.</li> <li>- Gestionar el cumplimiento del cronograma y objetivos.</li> <li>- Aprobar entregables y validar avances del proyecto</li> </ul> | <ul style="list-style-type: none"> <li>- Garantizar el correcto desarrollo del proyecto.</li> <li>- Resolver problemas organizacionales y técnicos.</li> <li>- Mantener comunicación con el cliente y el equipo.</li> </ul>                               |
| Gestor de Configuración de Software (CS) - Edu Sanchez |                                   | <ul style="list-style-type: none"> <li>- Administrar el control de versiones del proyecto.</li> <li>- Supervisar cambios en el software y documentación.</li> <li>- Gestionar repositorios y herramientas de integración.</li> </ul>                                                                    | <ul style="list-style-type: none"> <li>- Garantizar la integridad y trazabilidad de los cambios.</li> <li>- Mantener actualizado el repositorio del proyecto.</li> <li>- Coordinar el control de configuración del software.</li> </ul>                   |
| Bibliotecario - Max Rojas                              |                                   | <ul style="list-style-type: none"> <li>- Organizar y administrar la documentación del proyecto.</li> <li>- Mantener el registro de versiones de documentos.</li> <li>- Gestionar el almacenamiento de manuales, reportes y archivos técnicos.</li> </ul>                                                | <ul style="list-style-type: none"> <li>- Garantizar la disponibilidad y actualización de la documentación.</li> <li>- Controlar el acceso y distribución de documentos oficiales.</li> <li>- Mantener respaldo de la información del proyecto.</li> </ul> |
| Control de comité de cambios                           | Líder del Comité- Kevin Bohorquez | <ul style="list-style-type: none"> <li>- Coordinar las reuniones del comité de cambios.</li> <li>- Evaluar solicitudes de modificación del proyecto.</li> <li>- Supervisar el impacto de los cambios aprobados.</li> </ul>                                                                              | <ul style="list-style-type: none"> <li>- Autorizar cambios importantes en el sistema.</li> <li>- Garantizar que las modificaciones no afecten la estabilidad del proyecto.</li> </ul>                                                                     |
|                                                        | Arquitecto                        | <ul style="list-style-type: none"> <li>- Diseñar la arquitectura técnica del</li> </ul>                                                                                                                                                                                                                 | <ul style="list-style-type: none"> <li>- Garantizar la escalabilidad y</li> </ul>                                                                                                                                                                         |

|  |                                         |                                                                                                                                                           |                                                                                                                                        |
|--|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|  | técnico - Taco Miguel                   | sistema.<br>- Definir estándares tecnológicos y estructurales.<br>- Evaluar impactos técnicos de cambios solicitados.                                     | mantenibilidad del sistema.<br>- Supervisar la correcta integración de componentes.                                                    |
|  | Responsable de calidad - León Gerardo   | - Supervisar el cumplimiento de estándares de calidad.<br>- Planificar y validar pruebas del sistema.<br>- Elaborar reportes de control de calidad.       | - Garantizar la calidad del software antes de cada entrega.<br>- Verificar el cumplimiento de requisitos funcionales y no funcionales. |
|  | Desarrollador Backend - Sevan David     | - Implementar la lógica de negocio del sistema.<br>- Desarrollar APIs y servicios backend.<br>- Gestionar la conexión con la base de datos.               | - Garantizar el correcto funcionamiento del servidor.<br>- Optimizar rendimiento y seguridad del sistema.                              |
|  | Desarrollador Frontend - Lopez Leonardo | - Diseñar e implementar interfaces gráficas.<br>- Mejorar la experiencia de usuario.<br>- Integrar frontend con servicios backend.                        | - Garantizar accesibilidad y usabilidad del sistema.<br>- Mantener coherencia visual de la aplicación.                                 |
|  | Desarrollador DevOps - Edu Sanchez      | - Gestionar despliegues e integración continua.<br>- Administrar entornos de desarrollo y producción.<br>- Automatizar procesos de integración y entrega. | - Garantizar estabilidad y disponibilidad del sistema.<br>- Supervisar monitoreo y mantenimiento de servidores.                        |

### 3. Actividades de Gestión de Configuración

#### 3.1. Identificación de la Configuración

La Identificación de la Configuración es el proceso mediante el cual se seleccionan, nombran, describen y clasifican formalmente todos los elementos del proyecto que

serán objeto de control y seguimiento. Esta actividad establece la estructura base sobre la que operan los demás procesos de gestión de configuración.

Para el sistema Patitas Sanas, la identificación de la configuración abarca los artefactos producidos en el diseño, desarrollo, pruebas y despliegue del sistema de gestión clínica veterinaria, incluyendo tanto el software funcional como la documentación de soporte.

### 3.1.1. Catálogo de ECS del Proyecto Patitas Sanas

| #  | Categoría                | Elemento de Configuración (ECS)                             | Identificador |
|----|--------------------------|-------------------------------------------------------------|---------------|
| 1  | Código Fuente – Backend  | Módulo de gestión de mascotas (registro, historial clínico) | SCI-BE-001    |
| 2  | Código Fuente – Backend  | Módulo de veterinarios y especialidades                     | SCI-BE-002    |
| 3  | Código Fuente – Backend  | Módulo de turnos de trabajo                                 | SCI-BE-003    |
| 4  | Código Fuente – Backend  | Módulo de citas y agenda                                    | SCI-BE-004    |
| 5  | Código Fuente – Backend  | Módulo de autenticación y roles                             | SCI-BE-005    |
| 6  | Código Fuente – Backend  | API REST / controladores                                    | SCI-BE-006    |
| 7  | Código Fuente – Frontend | Pantalla de registro y perfil de mascotas                   | SCI-FE-001    |
| 8  | Código Fuente – Frontend | Panel de agenda y citas                                     | SCI-FE-002    |
| 9  | Código Fuente – Frontend | Panel de administración de veterinarios                     | SCI-FE-003    |
| 10 | Código Fuente – Frontend | Panel de turnos                                             | SCI-FE-004    |
| 11 | Base de Datos            | Scripts DDL (creación de esquema)                           | SCI-DB-001    |
| 12 | Base de Datos            | Scripts DML (datos semilla / seed)                          | SCI-DB-002    |
| 13 | Infraestructura          | Variables de entorno (.env.example)                         | SCI-INF-002   |
| 14 | Documentación            | Documento de Requerimientos del Software (SRS)              | SCI-DOC-001   |
| 15 | Documentación            | Plan de Gestión de Configuración                            | SCI-DOC-002   |
| 16 | Documentación            | Manuales de usuario y despliegue                            | SCI-DOC-003   |



### 3.1.2. Nomenclatura de Versiones

Todo ECS sujeto a control de versiones adoptará el estándar de Versionado Semántico (SemVer), ampliamente aceptado en la industria del software. El esquema es el siguiente:

| MAJOR (X)                                                 | MINOR (Y)                                   | PATCH (Z)              |
|-----------------------------------------------------------|---------------------------------------------|------------------------|
| Cambio que rompe compatibilidad con versiones anteriores. | Funcionalidad nueva compatible hacia atrás. | Corrección de errores. |

Formato completo: v{MAJOR}.{MINOR}.{PATCH}

| Ejemplo de Versión | Significado                                                        |
|--------------------|--------------------------------------------------------------------|
| v0.1.0             | Primera versión funcional interna (módulo de mascotas completado). |
| v0.2.0             | Se agrega el módulo de veterinarios y especialidades.              |
| v0.2.1             | Corrección de bug en el filtro de especialidades.                  |
| v1.0.0             | Primera línea base formal entregada al cliente / evaluador.        |
| v1.1.0             | Módulo de turnos de trabajo integrado.                             |

### 3.1.3 Estructura del Repositorio

El repositorio de Patitas Sanas en GitHub tendrá la siguiente estructura de directorios para garantizar que cada ECS esté ubicado en un lugar predecible y controlado:

| Ruta en el Repositorio | Contenido (ECS almacenados)                                   |
|------------------------|---------------------------------------------------------------|
| /backend/              | Código fuente del servidor (SCI-BE-001 al SCI-BE-006)         |
| /frontend/             | Código fuente de la interfaz (SCI-FE-001 al SCI-FE-004)       |
| /veterinariaDump       | Dump de la base de datos, estructura y registros (SCI-DB-001) |
| /docs/                 | Documentación del proyecto (SCI-DOC-001 al SCI-DOC-003)       |
| /.env.example          | Plantilla de variables de entorno (SCI-INF-002)               |

|               |                                            |
|---------------|--------------------------------------------|
| /README.md    | Guía rápida del proyecto y de contribución |
| /CHANGELOG.md | Registro histórico de cambios por versión  |

### 3.2. Control de la Configuración

El control de configuración del proyecto “Patitas Sanas” tiene como objetivo garantizar que los cambios realizados sobre el sistema sean gestionados de manera organizada, controlada y trazable durante el desarrollo del software. Para ello, se utilizará Git y GitHub como herramientas de control de versiones y administración del repositorio central. La gestión de configuración permitirá mantener la integridad del sistema y asegurar que las modificaciones realizadas sean registradas y validadas adecuadamente.

Todo cambio realizado en el proyecto deberá seguir un proceso de control previamente establecido. Las modificaciones podrán originarse por corrección de errores, implementación de nuevas funcionalidades, mejoras del sistema o ajustes solicitados durante el desarrollo. Cada solicitud de cambio será evaluada por el Comité de Control de Cambios (CCB), el cual determinará su aprobación y prioridad.

#### 3.2.1 Estrategia de Ramas

Para organizar el desarrollo del proyecto, se utilizará la siguiente estrategia de ramas:

| Tipo de Rama           | Patrón de Nombre            | Propósito                                                                      |
|------------------------|-----------------------------|--------------------------------------------------------------------------------|
| Principal              | main                        | Código en producción. Solo acepta merges desde release/*.                      |
| Integración            | develop                     | Rama de integración continua. Aquí se fusionan todas las features completadas. |
| Nueva funcionalidad    | feature/{descripcion-corta} | Desarrollo de un ECS nuevo.                                                    |
| Versión de lanzamiento | release/v{X}.{Y}.{Z}        | Preparación de una línea base.                                                 |

#### 3.2.2 Flujo de Control de Cambios

El flujo de control de cambios definido para el proyecto será el siguiente:

1. Registro y evaluación de la solicitud de cambio.

2. Creación de una rama feature/\* a partir de develop.
3. Desarrollo e implementación de los cambios correspondientes.
4. Registro de cambios mediante commits en el repositorio.
5. Creación de un Pull Request hacia la rama develop.
6. Revisión y validación del cambio por parte del Líder Técnico o responsable de QA.
7. Integración del cambio mediante merge hacia develop.
8. Creación de una rama release/v\* cuando las funcionalidades alcancen estabilidad.
9. Integración final de la versión aprobada hacia la rama master.

Con el fin de mantener el control y la estabilidad del proyecto, no se permitirán cambios directos sobre las ramas master y develop. Todo cambio deberá integrarse mediante Pull Requests y pasar por un proceso de revisión antes de su aprobación.

### 3.2.3 Convención de Commits

Los commits realizados en el proyecto deberán seguir una nomenclatura estándar con el fin de facilitar la trazabilidad y comprensión de los cambios realizados. Para ello, se utilizarán las siguientes etiquetas:

| Tipo     | Descripción                                   | Ejemplo                                                |
|----------|-----------------------------------------------|--------------------------------------------------------|
| feat     | Nueva funcionalidad.                          | feat(citas): agregar filtro por especialidad           |
| fix      | Corrección de un bug.                         | fix(turnos): corregir solapamiento de horarios         |
| docs     | Cambios solo en documentación.                | docs(readme): actualizar instrucciones de despliegue   |
| refactor | Refactorización sin cambio de comportamiento. | refactor(veterinarios): simplificar lógica de búsqueda |
| test     | Agregar o modificar pruebas.                  | test(mascotas): agregar prueba de registro duplicado   |
| chore    | Tareas de mantenimiento.                      | chore(deps): actualizar dependencias npm               |

Ejemplos:

- ❖ feat: agregar módulo de citas
- ❖ fix: corregir validación de login
- ❖ docs: actualizar documentación del backend

### 3.3. Contabilidad del Estado de Configuración

La Contabilidad del Estado de Configuración registra y reporta el estado de cada Elemento de Configuración (ECS), las solicitudes de cambio y el avance de las líneas base del proyecto Patitas Sanas.

El responsable de esta actividad es el Gestor de Configuración de Software, quien consolida la información proveniente de Jira y GitHub.

#### 3.3.1. Información a registrar por cada ECS

| Atributo            | Descripción                                           |
|---------------------|-------------------------------------------------------|
| Identificador       | Código del ECS según el catálogo (p. ej. SCI-BE-001). |
| Versión vigente     | Versión actual bajo el esquema SemVer.                |
| Línea Base          | Baseline al que pertenece la versión actual.          |
| Estado              | En desarrollo / En revisión / Aprobado / Liberado.    |
| Responsable         | Miembro del equipo a cargo del elemento.              |
| Última modificación | Fecha y referencia al commit o Pull Request.          |

#### 3.3.2. Registros oficiales

| Registro               | Herramienta                              | Contenido                              |
|------------------------|------------------------------------------|----------------------------------------|
| Estado de ECS          | Hoja en<br>/docs/control/estado_ecs.xlsx | Versión y estado vigente de cada SCI.  |
| Solicitudes de Cambio  | Jira (tipo <i>Change Request</i> )       | Decisión del CCB y trazabilidad al PR. |
| Histórico de Versiones | CHANGELOG.md                             | Cambios consolidados por versión.      |

#### 3.3.3. Estados de una Solicitud de Cambio

| Estado        | Descripción                                         |
|---------------|-----------------------------------------------------|
| Registrada    | Creada en Jira por un miembro del equipo o cliente. |
| En evaluación | El CCB evalúa impacto y prioridad.                  |
| Aprobada      | Autorizada por el CCB.                              |
| Rechazada     | No procede; se documenta la justificación.          |

#### 3.3.4. Reportes

| Reporte              | Frecuencia              | Contenido                                                                       |
|----------------------|-------------------------|---------------------------------------------------------------------------------|
| Reporte semanal      | Cada viernes            | Solicitudes cerradas, abiertas y diferidas; avance hacia la próxima línea base. |
| Cierre de Línea Base | Al cerrar cada baseline | ECS incluidos, solicitudes integradas, bugs resueltos y diferidos.              |

#### 3.4. Auditoria y revisiones de configuración

Se realizarán auditorías de configuración antes de la liberación de cada línea base o de una actualización de versión de un componente prioritario. Su objetivo es verificar que la línea base esté compuesta por una colección consistente y bien definida de productos.

##### 3.4.1. Elementos de Configuración bajo Auditoría

Se seleccionarán uno o más ECS de mayor prioridad en la línea base, priorizando aquellos con mayor complejidad técnica, criticidad funcional o mayor número de cambios registrados en Jira.

##### 3.4.2. Agenda de Auditorías

| Auditoría | Línea Base                             | Semana    | ECS Prioritarios                           |
|-----------|----------------------------------------|-----------|--------------------------------------------|
| AUD-01    | LB1 – Módulo Administrativo            | Semana 3  | SCI-BE-005, SCI-BE-002, SCI-FE-003         |
| AUD-02    | LB2 – Recepción y Pacientes            | Semana 5  | SCI-BE-001, SCI-BE-004, SCI-FE-001         |
| AUD-03    | LB3 – Core Clínico                     | Semana 7  | SCI-BE-001, SCI-BE-003, SCI-BE-006         |
| AUD-04    | LB4 – Reportes e Inteligencia de Datos | Semana 9  | SCI-DB-001, SCI-FE-004                     |
| AUD-05    | LB5 – Release Final                    | Semana 10 | Todos los ECS + SCI-DOC-001 al SCI-DOC-003 |

### 3.4.3. Conducción y Participantes

Las auditorías serán dirigidas por el SCMR (Edu Sanchez). Los participantes obligatorios son el SCMR y los autores de los ECS a auditar; el Responsable de QA (León Gerardo) y el Líder del CCB (Kevin Bohorquez) participan de forma recomendada.

### 3.4.4. Documentos Requeridos

- Documentos de Solicitudes de Cambio (SCR) registrados en Jira.
- Reporte de estado de configuración: estado\_ecs.xlsx en /docs/control/.
- CHANGELOG.md y Pull Requests cerrados en GitHub.

### 3.4.5. Reporte de Deficiencias y Acciones Correctivas

Las deficiencias identificadas serán documentadas por los participantes durante la auditoría, registrando el ECS afectado, la descripción del problema y la acción correctiva acordada. Cada deficiencia será cargada como tarea en Jira y monitoreada hasta su cierre por el SCMR.

### 3.4.6. Criterio de Aprobación

El criterio de aprobación es determinado por el SCMR. Una línea base será aprobada cuando no existan deficiencias críticas pendientes, todos los ECS auditados presenten la versión correcta, y los cambios aprobados por el CCB estén reflejados en el CHANGELOG.md. Las deficiencias menores deben contar con un plan de acción registrado en Jira.

## 4. Cronograma y Líneas Base (Baselines)

### 4.1. Definición de Líneas Base (Baselines)

#### **Fase 1 (Semana 3) - "Fundación y Módulo Administrativo":**

Se congela el código tras finalizar la configuración de la base de datos (MySQL), la arquitectura base (FastAPI y React) y el Módulo del Administrador. Esta versión incluye el CRUD de usuarios, control de roles, catálogo de servicios y configuración de turnos.

### **Fase 2 (Semana 5) - "Flujo de Recepción y Pacientes":**

Se establece al finalizar el Módulo de la Recepcionista. Esta versión estable incluye la gestión completa de clientes, registro de mascotas, solicitudes de atención inmediata y el sistema de agendamiento de citas.

### **Fase 3 (Semana 7) - "Core Clínico":**

Se congela el código al culminar el Módulo del Veterinario. Esta versión integra el registro de triaje, la gestión de consultas médicas (diagnósticos y tratamientos) y el acceso al Historial Clínico Digital.

### **Fase 4 (Semana 9) - "Inteligencia de Datos y Reportes":**

Se establece al integrar todos los Procedimientos Almacenados (Stored Procedures) y la visualización de los reportes estratégicos (Resumen diario, ranking de patologías y atenciones).

### **Fase 5 (Semana 10) - "Release Final (Producción)" - Línea base 1:**

Versión final, estable y desplegada en Railway, con todas las pruebas de integración superadas, corrección de errores (bug fixing) finalizada y documentación técnica completada

## **4.2. Cronograma de Desarrollo (10 Semanas)**

A continuación, se detalla la planificación semanal del proyecto orientada a cumplir con las líneas base establecidas:

### **Fase 1: Planificación y Arquitectura**

**Semana 1:** Levantamiento de requerimientos, diseño del modelo relacional (Base de Datos) y configuración de repositorios (Git/GitHub).

**Semana 2:** Creación de la API base en FastAPI, conexión a MySQL y diseño de la interfaz (UI/UX) inicial en React.

### **Fase 2: Desarrollo del Módulo Administrativo**

**Semana 3:** Implementación del sistema de login/autenticación, gestión de personal, roles, servicios y turnos. *(Cierre: Línea Base 1).*

### **Fase 3: Desarrollo del Módulo de Recepción**

**Semana 4:** Desarrollo del registro relacional de Clientes y Mascotas.

**Semana 5:** Implementación del flujo de solicitudes de atención y agendamiento de citas programadas. (*Cierre: Línea Base 2*).

#### **Fase 4: Desarrollo del Módulo Veterinario**

**Semana 6:** Interfaz de cola de pacientes para el veterinario y formulario de signos vitales (Triaje).

**Semana 7:** Registro de consultas médicas y visualización del Historial Clínico. (*Cierre: Línea Base 3*).

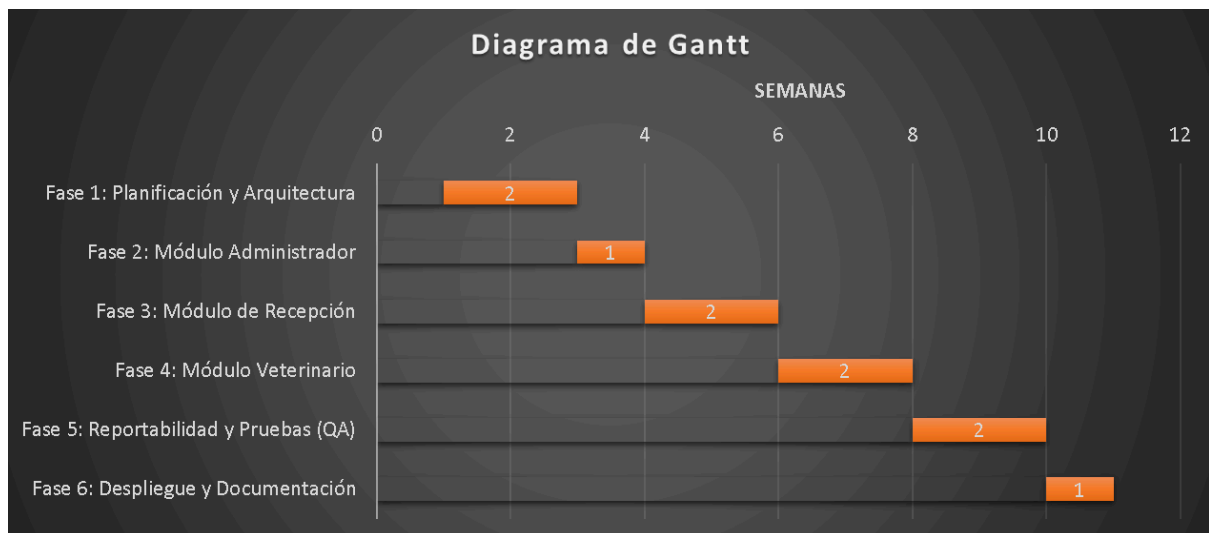
#### **Fase 5: Reportabilidad y Ajustes**

**Semana 8:** Creación de *Stored Procedures* en MySQL para consultas complejas.

**Semana 9:** Integración de los Dashboards y Reportes en el frontend. Fase intensiva de pruebas (QA). (*Cierre: Línea Base 4*).

#### **Fase 6: Despliegue y Entrega**

**Semana 10:** Despliegue del backend y base de datos en Railway. Elaboración de manuales y presentación final. (*Cierre: Línea Base 5*).





## 5. Herramientas y Entorno

Para garantizar la integridad, trazabilidad y correcta gestión de los Elementos de Configuración (SCI) del proyecto, se ha definido una arquitectura de herramientas basada en estándares de la industria y repositorios distribuidos.

### 5.1. Sistema control de versiones

El control de versiones se gestionará de manera distribuida utilizando **Git**. Esta herramienta será la encargada de mantener el historial local de los cambios en el código fuente y en la documentación técnica.

### 5.2. Repositorio central

Como plataforma de alojamiento y colaboración principal, se utilizará **GitHub**. Se mantendrán un solo repositorio para los 2 componentes de arquitectura del sistema (backend y frontend) además de la descripción del proyecto *README.md*.

### 5.3. Entornos de ejecución y despliegue

- **Entorno Cliente:** El software está diseñado para ejecutarse en navegadores web modernos (Google Chrome, Mozilla Firefox, Microsoft Edge).
- **Paridad de Entornos y Hosting:** Para el despliegue final, se utilizará una plataforma como servicio (**Railway**) para el backend, (**Railway**) para la base de datos y (**Vercel**) para el frontend.

### 5.4. Stack tecnológico

- **Frontend:** Desarrollado en **JavaScript** utilizando la biblioteca **React**. Las dependencias y versiones de paquetes externos serán gestionadas estrictamente a través del archivo **package.json** utilizando **npm**.
- **Backend (Servidor):** Desarrollado en **Python** utilizando el framework **FastApi**. Las dependencias del servidor se controlarán fijando las versiones exactas en el archivo **requirements.txt** utilizando **pip**.

### 5.5. Gestión de base de datos

La persistencia de datos y sus cambios estructurales son elementos críticos del sistema. Se utilizará **MySQL** como sistema de gestión de bases de datos relacionales.

### 5.6. Documentación Integrado

Para asegurar que la documentación técnica y de gestión evolucione a la par del sistema, se adoptará una estrategia de **documentación integrada directamente en el repositorio del proyecto**.

## 5.7. Comunicación y Reportes de Estado

Para asegurar que todos los interesados (desarrolladores, SCMR y dirección) estén alineados, se establecen dos niveles de comunicación:

- Se utilizará **Jira** como la herramienta oficial para el control del estado. Cada asignación de tareas y reportes de progreso se registran aquí para garantizar la trazabilidad y auditoría del software.
- Se define **WhatsApp** como canal principal para comunicaciones rápidas, alertas urgentes y coordinación logística entre los miembros del equipo.

## Referencias

**INACAL (2016).** NTP-ISO/IEC 12207:2016 Ingeniería de Software y Sistemas. Procesos del ciclo de vida del software.

**Zeballos, Y. (2009).** *K2BIM: Plan de Configuración Versión 0.9*. Calaméo. Recuperado de <https://www.calameo.com/read/004372137e7b3f511aa5f>